

Assignment Report

Big-M Simplex Method

Transportation Problem — VAM and MODI

Computational solutions (Python 3)

This report solves two well-known constrained optimization problems computationally. Each method is implemented from first principles (no NumPy / SciPy solver). The submitted PDF includes the Python source and the corresponding program output for Big-M, Vogel's Approximation Method (VAM), and the Modified Distribution (MODI) method.

Part 1. Big-M Method. A mixed-constraint linear program is converted to standard form with slack, surplus and artificial variables, then solved by Big-M simplex. Exact a + bM arithmetic is used throughout.

Part 2. Transportation Problem. A balanced 3-factory / 4-warehouse problem is solved in two stages, one method at a time: (i) VAM constructs an initial basic feasible solution; (ii) MODI tests optimality and improves the allocation until every opportunity cost is non-negative.

Part	Method	Program	Result
1	Big-M simplex	big_m_method.py	$Z^* = 17/5 = 3.4$
2(i)	VAM (IBFS)	transportation.py --method vam	cost = 779
2(ii)	MODI (optimal)	transportation.py --method modi	$Z^* = 743$

How to reproduce:

```
python3 big_m_method.py
```

```
python3 transportation.py --method vam
```

```
python3 transportation.py --method modi
```

Part 1 — Big-M Simplex Method

Ordinary simplex needs an initial basic feasible solution. That is immediate when every constraint is of type $\leq$ (slack variables form the basis). Equality and $\geq$ constraints require artificial variables. Big-M places a huge penalty M on each artificial in the objective so simplex drives them to zero whenever the LPP is feasible.

If an artificial remains positive at termination, the original LPP is infeasible.

Selected LPP (Taha / Winston mixed-constraint example)

Minimize $Z = 4x_1 + x_2$

subject to $3x_1 + x_2 = 3$

$4x_1 + 3x_2 \geq 6$

$x_1 + 2x_2 \leq 4$

$x_1, x_2 \geq 0$

All three constraint types appear, so slack, surplus and artificial variables are required.

Standard form

$3x_1 + x_2 + a_1 = 3$

$4x_1 + 3x_2 - p_1 + a_2 = 6$

$x_1 + 2x_2 + s_1 = 4$

$Z = 4x_1 + x_2 + M(a_1 + a_2)$ (minimization: artificials are expensive)

Initial basis: $\{a_1, a_2, s_1\}$. The program stores every tableau entry as $a + bM$ using exact fractions.

Simplex rules used

1. Convert to standard form; put artificials / slacks in the basis.
2. Compute reduced costs $c_j - z_j$ after eliminating basic variables from Z .
3. MIN optimality test: stop when every $c_j - z_j \geq 0$.
4. Entering variable: most negative $c_j - z_j$.
5. Leaving variable: minimum-ratio test (smallest non-negative RHS / pivot column).
6. Pivot and repeat.

Part 1 — Iterations and optimal solution

Iter.	Basis	Z	Enter	Leave
0	a1, a2, s1	9M	x1 (4-7M)	a1
1	x1, a2, s1	4+2M	x2 (-1/3-5/3 M)	a2
2	x1, x2, s1	18/5	p1 (-1/5)	s1
3	x1, x2, p1	17/5	— optimal	—

Both artificials have left the basis, so the solution is feasible for the original LPP.

Optimal solution

Variable	Value	Comment
x1*	$2/5 = 0.4$	decision
x2*	$9/5 = 1.8$	decision
p1	1	surplus on the $\geq$ constraint
s1, a1, a2	0	artificials must be zero
Z*	$17/5 = 3.4$	minimum cost

Verification

$$3(0.4) + 1.8 = 3$$

$$4(0.4) + 3(1.8) = 7 \geq 6$$

$$0.4 + 2(1.8) = 4$$

$$Z = 4(0.4) + 1.8 = 3.4$$

The next pages are screenshots of big_m_method.py and of the terminal output produced by: `python3 big_m_method.py`

Source file big_m_method.py (run with: python3 big_m_method.py)

```

1  """
2  Big-M Simplex Method
3  =====
4  Solves a classic mixed-constraint Linear Programming Problem (LPP)
5  by converting it to standard form (slack / surplus / artificial variables)
6  and applying the Big-M simplex algorithm.
7
8  Textbook problem (Taha / Winston):
9  Minimise   $Z = 4x_1 + x_2$ 
10 subject to
11      $2x_1 + x_2 = 3$ 
12      $4x_1 + 3x_2 \geq 6$ 
13      $x_1 + 2x_2 \leq 4$ 
14      $x_1, x_2 \geq 0$ 
15 """
16
17 from fractions import Fraction
18 from typing import List, Optional, Sequence, Tuple
19
20
21 # -----
22 # Arithmetic: every tableau entry is (constant) + (coeff) * M
23 # -----
24 M_SYMBOL = "M"
25 Value = Tuple[Fraction, Fraction] # (const, m_coeff) => const + m_coeff * M
26
27
28 def V(const=0, m=0) -> Value:
29     return (Fraction(const), Fraction(m))
30
31
32 def add(a: Value, b: Value) -> Value:
33     return (a[0] + b[0], a[1] + b[1])
34
35
36 def sub(a: Value, b: Value) -> Value:
37     return (a[0] - b[0], a[1] - b[1])
38
39
40 def mul(a: Value, k: Fraction) -> Value:
41     return (a[0] * k, a[1] * k)
42
43
44 def fnc(v: Value) -> str:
45     c, m = v
46     if m == 0:
47         return _frac(c)
48     if c == 0:
49         if m == 1:
50             return M_SYMBOL
51         if m == -1:
52             return f"~{M_SYMBOL}"
53         return f"({_frac(m)}){M_SYMBOL}"
54     m_part = M_SYMBOL if m == 1 else (f"~{M_SYMBOL}" if m == -1 else f"({_frac(abs(m)})}{M_SYMBOL}")
55     if m > 0:
56         return f"({_frac(c)}){m_part}"
57     return f"({_frac(c)})~{m_part}"
58
59
60 def _frac(x: Fraction) -> str:
61     if x.denominator == 1:
62         return str(x.numerator)

```

```

63     return f"{n.numerator}/{n.denominator}"
64
65
66 def is_neg(v: Value) -> bool:
67     """True iff v is strictly negative for a sufficiently large M > 0."""
68     c, m = v
69     if m < 0:
70         return True
71     if m > 0:
72         return False
73     return c < 0
74
75
76 def cmp_more_neg(a: Value, b: Value) -> bool:
77     """True iff a < b for large M (used to pick the most negative reduced cost)."""
78     dc, dm = sub(a, b)
79     if dm != 0:
80         return dm < 0
81     return dc < 0
82
83
84 # -----
85 # Pretty printing
86 # -----
87 def _hline(widths: Sequence[int], left="+", mid="=", right="+") -> str:
88     return left + mid.join("-" * (w + 2) for w in widths) + right
89
90
91 def _row(cells: Sequence[str], widths: Sequence[int]) -> str:
92     return "|" + "|".join(f"{c:^{w}}" for c, w in zip(cells, widths)) + "|"
93
94
95 def print_table(headers: Sequence[str], rows: Sequence[Sequence[str]], title: str = "") -> None:
96     widths = [len(h) for h in headers]
97     for row in rows:
98         for i, cell in enumerate(row):
99             widths[i] = max(widths[i], len(str(cell)))
100
101     if title:
102         print(f"\n{title}")
103     print(_hline(widths))
104     print(_row(headers, widths))
105     print(_hline(widths))
106     for row in rows:
107         print(_row([str(c) for c in row], widths))
108     print(_hline(widths))
109
110 # -----
111 # Big-M Simplex solver
112 # -----
113 class BigMSimplex:
114     """
115     Tableau Big-M simplex.
116
117     constraints: list of (coeffs, sense, rhs)
118         sense in {'<=', '>=', '=' }
119     sense_obj: 'min' or 'max'
120     """
121
122     def __init__(
123         self,
124         c: Sequence[float],

```

```

big_m_method.py — Python source

125     constraints: Sequence[Tuple[Sequence[float], str, float]],
126     sense: str = "min",
127     var_names: Optional[Sequence[str]] = None,
128 ):
129     self.orig_c = [Fraction(n) for n in c]
130     self.n_orig = len(c)
131     self.sense = sense.lower()
132     self.var_names = list(var_names) if var_names else [f"x{i+1}" for i in range(self.n_orig)]
133     self.constraints = [
134         ((Fraction(a) for a in coeffs), s, Fraction(rhs)) for coeffs, s, rhs in constraints
135     ]
136
137     self.col_names: List[str] = []
138     self.A: List[List[Value]] = [] # constraints rows (no s-row)
139     self.b: List[Value] = []
140     self.c_row: List[Value] = [] # original objective coefficients (c_j)
141     self.basis: List[int] = []
142     self.row_names: List[str] = []
143     self.artificial_idx: List[int] = []
144     self.s0: Value = V(0)
145
146     $ ----- Formulation -----
147     def to_standard_form(self) -> None:
148         n = self.n_orig
149         slack_k = surplus_k = art_k = 0
150
151         $ Count extra variables so names / columns are stable.
152         extras: List[Tuple[str, int, str]] = [] # (kind, constraint_i, name)
153         for i, (_, sense, _) in enumerate(self.constraints):
154             if sense == "<=":
155                 slack_k += 1
156                 extras.append(("slack", i, f"s{slack_k}"))
157             elif sense == ">=":
158                 surplus_k += 1
159                 art_k += 1
160                 extras.append(("surplus", i, f"p{surplus_k}"))
161                 extras.append(("art", i, f"a{art_k}"))
162             elif sense == "=":
163                 art_k += 1
164                 extras.append(("art", i, f"a{art_k}"))
165             else:
166                 raise ValueError(f"Unknown sense {sense}")
167
168         self.col_names = list(self.var_names) + [name for _, _, name in extras]
169         n_cols = len(self.col_names)
170
171         $ Original c_j (decision vars). Slack/surplus get 0.
172         $ Artificial: +M if minimize, -M if maximize.
173         self.c_row = [V(0) for _ in range(n_cols)]
174         for j in range(n):
175             self.c_row[j] = V(self.orig_c[j], 0)
176         extra_pos = n
177         for kind, _, name in extras:
178             if kind == "art":
179                 m_pen = Fraction(1) if self.sense == "min" else Fraction(-1)
180                 self.c_row[extra_pos] = V(0, m_pen)
181                 self.artificial_idx.append(extra_pos)
182                 extra_pos += 1
183
184         $ Build constraint matrix
185         extra_pos = n
186         extra_iter = iter(extras)

```

```

big_m_method.py — Python source

187     self.A = []
188     self.b = []
189     self.basis = []
190     self.row_names = []
191
192     for i, (coeffs, sense, rhs) in enumerate(self.constraints):
193         if rhs < 0:
194             coeffs = [-a for a in coeffs]
195             rhs = -rhs
196             sense = (">=" if "<=" in sense else "<=" if ">=" in sense else "<=")
197
198         row = [V(0) for _ in range(n_cols)]
199         for j in range(n):
200             row[j] = V(coeffs[j])
201
202         if sense == "<=":
203             kind, _, name = next(extra_iter)
204             assert kind == "slack"
205             row[extra_pos] = V(1)
206             self.basis.append(extra_pos)
207             self.row_names.append(name)
208             extra_pos += 1
209         elif sense == ">=":
210             kind, _, name = next(extra_iter)
211             assert kind == "surplus"
212             row[extra_pos] = V(-1)
213             extra_pos += 1
214             kind, _, name = next(extra_iter)
215             assert kind == "art"
216             row[extra_pos] = V(1)
217             self.basis.append(extra_pos)
218             self.row_names.append(name)
219             extra_pos += 1
220         else:  # "="
221             kind, _, name = next(extra_iter)
222             assert kind == "art"
223             row[extra_pos] = V(1)
224             self.basis.append(extra_pos)
225             self.row_names.append(name)
226             extra_pos += 1
227
228         self.A.append(row)
229         self.b.append(V(rhs))
230
231     self._recompute_cj_cj()
232
233     def _recompute_cj_cj(self) -> None:
234         n_cols = len(self.col_names)
235         cj = [V(0) for _ in range(n_cols)]
236         z_val = V(0)
237         for i, bidx in enumerate(self.basis):
238             cb = self.c_row[bidx]
239             for j in range(n_cols):
240                 cj[j] = add(cj[j], _mul_val(cb, self.A[i][j]))
241             z_val = add(z_val, _mul_val(cb, self.b[i]))
242         self.cj_cj = [sub(self.c_row[j], cj[j]) for j in range(n_cols)]
243         self.z0 = z_val
244
245     # ----- tableau I/O -----
246     def print_formulation(self) -> None:
247         obj = " + ".join(
248             f"[{frac(self.orig_c[j])}(self.var_names[j])]" if self.orig_c[j] != 1

```

```

big_m_method.py — Python source

249         else self.var_names[j]
250         for j in range(self.n_orig)
251     )
252     print("=" * 72)
253     print("ORIGINAL LINEAR PROGRAMMING PROBLEM")
254     print("=" * 72)
255     print(f"    (self.sense.upper())IMIZE Z = (obj)")
256     print("    subject to")
257     for coeffs, sense, rhs in self.constraints:
258         terms = []
259         for j, a in enumerate(coeffs):
260             if a == 0:
261                 continue
262             terms.append(f"({_frac(a)}{self.var_names[j]})" if a != 1 else self.var_names[j])
263         print(f"    (' + '.join(terms)) (sense) ({_frac(rhs)})")
264         print("    " + ", ".join(self.var_names) + " >= 0")
265
266 def print_standard_form(self) -> None:
267     print("\n" + "=" * 72)
268     print("STANDARD FORM (slack / surplus / artificial variables)")
269     print("=" * 72)
270     print(" Slack (s): added to <= constraints")
271     print(" Surplus (p): subtracted from >= constraints")
272     print(" Artificial(a): added to >= and = constraints")
273     print()
274     if self.sense == "min":
275         print(" Big-M objective (MIN): Z = original cost + M * (sum of artificials)")
276     else:
277         print(" Big-M objective (MAX): Z = original profit - M * (sum of artificials)")
278     print()
279
280 n = self.n_orig
281 for i, (coeffs, sense, rhs) in enumerate(self.constraints):
282     parts = []
283     for j in range(len(self.col_names)):
284         coeff = self.A[i][j]
285         if coeff == 0:
286             continue
287         name = self.col_names[j]
288         s = f"{coeff}"
289         if s == "1":
290             parts.append(name)
291         elif s == "-1":
292             parts.append(f"- {name}")
293         else:
294             parts.append(f"{s}{name}")
295     pretty = parts[0]
296     for p in parts[1:]:
297         if p.startswith("- "):
298             pretty += " - " + p[2:]
299         else:
300             pretty += " + " + p
301     print(f"    {pretty} = {f'{self.b[i]}'")
302
303 art_names = [self.col_names[j] for j in self.artificial_ids]
304 print()
305 print(f" Artificial variables: {' , '.join(art_names) if art_names else '(none)'}")
306 print(f" Initial basis:      {' , '.join(self.row_names)}")
307
308 def print_tableau(self, iteration: int) -> None:
309     headers = ["BV", "cb"] + self.col_names + ["RHS"]
310     rows = []

```

```

big_m_method.py — Python source

211     for i, bidx in enumerate(self.basis):
212         rows.append(
213             [self.col_names[bidx], fmt(self.c_row[bidx])]
214             + [fmt(self.A[i][j]) for j in range(len(self.col_names))]
215             + [fmt(self.b[i])]
216         )
217     rows.append(
218         ["cj - zj", ""]
219         + [fmt(v) for v in self.cj_cj]
220         + [fmt(self.s0)]
221     )
222     title = f"SIMPLEX TABLEAU - Iteration {iteration} (current Z = {fmt(self.s0)})"
223     print_table(headers, rows, title)
224
225     $ ----- simplex iterations -----
226     def _entering(self) -> Optional[int]:
227         best = None
228         best_val = None
229         for j in range(len(self.col_names)):
230             v = self.cj_cj[j]
231             if self.sense == "min":
232                 $ most negative cj-zj
233                 if is_neg(v) and (best is None or cmp_more_neg(v, best_val)):
234                     best, best_val = j, v
235             else:
236                 $ most positive cj-zj <=> most negative -(cj-zj)
237                 nv = mul(v, Fraction(-1))
238                 if is_neg(nv) and (best is None or cmp_more_neg(nv, mul(best_val, Fraction(-1)))):
239                     best, best_val = j, v
240         return best
241
242     def _leaving(self, enter: int) -> Optional[int]:
243         best_i = None
244         best_ratio = None
245         print("\n Minimum-ratio test (leaving variable):")
246         print(f" ('BV':<8) ('RHS':<12) ('pivot col':<12) ('ratio':<12)")
247         for i in range(len(self.A)):
248             aij = self.A[i][enter]
249             $ pivot column must be strictly positive (no M in constraints here)
250             if aij[1] != 0 or aij[0] <= 0:
251                 print(f" {self.col_names[self.basis[i]]:<8} {fmt(self.b[i]):<12} {fmt(aij):<12} {'---':<12}")
252                 continue
253             ratio = self.b[i][0] / aij[0]
254             print(f" {self.col_names[self.basis[i]]:<8} {fmt(self.b[i]):<12} {fmt(aij):<12} {_frac(ratio):<12}")
255             if best_i is None or ratio < best_ratio:
256                 best_i, best_ratio = i, ratio
257         return best_i
258
259     def _pivot(self, leave_row: int, enter_col: int) -> None:
260         piv = self.A[leave_row][enter_col]
261         assert piv[1] == 0 and piv[0] != 0
262         inv = Fraction(1) / piv[0]
263         m = len(self.A)
264         n = len(self.col_names)
265
266         self.A[leave_row] = [mul(self.A[leave_row][j], inv) for j in range(n)]
267         self.b[leave_row] = mul(self.b[leave_row], inv)
268
269         for i in range(m):
270             if i == leave_row:
271                 continue
272             factor = self.A[i][enter_col]

```

```

big_m_method.py — Python source

273     self.A[i] = [sub(self.A[i][j], _mul_val(factor, self.A[leave_row][j])) for j in range(n)]
274     self.b[i] = sub(self.b[i], _mul_val(factor, self.b[leave_row]))
275
276     self.basis[leave_row] = enter_col
277     self.row_names[leave_row] = self.col_names[enter_col]
278     self._recompute_cj_sj()
279
280 def solve(self, max_iter: int = 50) -> None:
281     self.print_formulation()
282     self.to_standard_form()
283     self.print_standard_form()
284
285     print("\n" + "=" * 72)
286     print("BIG-M SIMPLEX ITERATIONS")
287     print("=" * 72)
288     print(" Optimality test:")
289     if self.sense == "Min":
290         print(" MIN problem - stop when every (cj - sj) >= 0")
291         print(" Entering variable: most negative (cj - sj)")
292     else:
293         print(" MAX problem - stop when every (cj - sj) <= 0")
294         print(" Entering variable: most positive (cj - sj)")
295     print(" Leaving variable: minimum non-negative RHS / pivot-column ratio")
296
297     for it in range(max_iter + 1):
298         self.print_tableau(it)
299
300         enter = self._entering()
301         if enter is None:
302             print("\n All reduced costs satisfy the optimality condition. STOP.")
303             self._report()
304             return
305
306         print(f"\n Entering variable: {self.col_names[enter]} (cj - sj = {fmt(self.cj_sj[enter])})")
307         leave = self._leaving(enter)
308         if leave is None:
309             print("\n Unbounded problem (no positive pivot).")
310             return
311         print(f" Leaving variable: {self.col_names[self.basis[leave]]}")
312         print(f" Pivot element: {fmt(self.A[leave][enter])} at ({self.col_names[self.basis[leave]]}, {self.col_names[enter]})")
313         self._pivot(leave, enter)
314
315     print("Iteration limits reached.")
316
317 def _report(self) -> None:
318     print("\n" + "=" * 72)
319     print("OPTIMAL SOLUTION")
320     print("=" * 72)
321
322     values = {name: Fraction(0) for name in self.var_names}
323     art_values = {}
324     all_vals = {name: Fraction(0) for name in self.col_names}
325     for i, bidx in enumerate(self.basis):
326         name = self.col_names[bidx]
327         val = self.b[i]
328         if val[i] != 0:
329             print(f" WARNING: basic variable {name} still contains M.")
330         all_vals[name] = val[i]
331         if name in values:
332             values[name] = val[i]
333         if bidx in self.artificial_idx:
334             art_values[name] = val[i]

```

```

435
436     infeasible = any(v != 0 for v in art_values.values())
437     if infeasible:
438         print(" INFEASIBLE: a positive artificial variable remains in the basis.")
439         for n, v in art_values.items():
440             print(f"      {n} = {_frac(v)}")
441         return
442
443     print(" Decision variables:")
444     for name in self.var_names:
445         print(f"      {name}* = {_frac(values[name])}    {(float(values[name]):.4f)}")
446
447     print(" Slack / surplus / artificial (all artificials must be 0):")
448     for name in self.col_names[self.n_orig :]:
449         print(f"      {name} = {_frac(all_vals[name])}")
450
451     z = self.z0[0] # M-part must be 0 at a feasible optimum
452     print()
453     print(f" Optimal objective value: Z* = {_frac(z)}    {(float(z):.4f)}")
454     print(" " * 72)
455
456
457 def _mul_val(a: Value, b: Value) -> Value:
458     """Product of two (const + m M) values. Constraint entries have m=0."""
459     return (a[0] * b[0], a[0] * b[1] + a[1] * b[0])
460
461
462 def main() -> None:
463     print("\n\nASSIGNMENT - BIG-M SIMPLEX METHOD\n\n")
464     solver = BigMSimplex(
465         c=[4, 1],
466         constraints=[
467             ((3, 1), "=", 3),
468             ((14, 3), ">=", 6),
469             ((1, 2), "<=", 4),
470         ],
471         sense="min",
472         var_names=["x1", "x2"],
473     )
474     solver.solve()
475
476
477 if __name__ == "__main__":
478     main()

```

Captured terminal output of the program (screenshot-style).

```

Terminal — python3 big_m_method.py

ASSIGNMENT - BIG-M SIMPLEX METHOD

=====
ORIGINAL LINEAR PROGRAMMING PROBLEM
=====
MINIMIZE  $Z = 4x_1 + x_2$ 
subject to
 $3x_1 + x_2 = 3$ 
 $4x_1 + 3x_2 \geq 6$ 
 $x_1 + 2x_2 \leq 4$ 
 $x_1, x_2 \geq 0$ 

=====
STANDARD FORM (slack / surplus / artificial variables)
=====
Slack (s): added to  $\leq$  constraints
Surplus (p): subtracted from  $\geq$  constraints
Artificial(a): added to  $\geq$  and  $=$  constraints

Big-M objective (MIN):  $Z = \text{original cost} + M * (\text{sum of artificials})$ 

 $3x_1 + x_2 + a_1 = 3$ 
 $4x_1 + 3x_2 - p_1 + a_2 = 6$ 
 $x_1 + 2x_2 + s_1 = 4$ 

Artificial variables:  $a_1, a_2$ 
Initial basis:  $a_1, a_2, s_1$ 

=====
BIG-M SIMPLEX ITERATIONS
=====
Optimality test:
MIN problem - stop when every  $(c_j - s_j) \geq 0$ 
Entering variable: most negative  $(c_j - s_j)$ 
Leaving variable: minimum non-negative RHS / pivot-column ratio

SIMPLEX TABLEAU - Iteration 0 (current  $Z = 9M$ )
=====
| BV | cb |  $x_1$  |  $x_2$  |  $a_1$  |  $p_1$  |  $a_2$  |  $s_1$  | RHS |
=====
|  $a_1$  | M | 3 | 1 | 1 | 0 | 0 | 0 | 3 |
|  $a_2$  | M | 4 | 3 | 0 | -1 | 1 | 0 | 6 |
|  $s_1$  | 0 | 1 | 2 | 0 | 0 | 0 | 1 | 4 |
|  $c_j - s_j$  | | 4-7M | 1-4M | 0 | M | 0 | 0 | 9M |
=====

Entering variable:  $x_1$  ( $c_j - s_j = 4-7M$ )

Minimum-ratio test (leaving variable):
BV RHS pivot col ratio
 $a_1$  3 3 1
 $a_2$  6 4 3/2
 $s_1$  4 1 4
Leaving variable:  $a_1$ 
Pivot element: 3 at ( $a_1, x_1$ )

SIMPLEX TABLEAU - Iteration 1 (current  $Z = 4+2M$ )
=====
| BV | cb |  $x_1$  |  $x_2$  |  $a_1$  |  $p_1$  |  $a_2$  |  $s_1$  | RHS |
=====
|  $x_1$  | 4 | 1 | 1/3 | 1/3 | 0 | 0 | 0 | 1 |
|  $a_2$  | M | 0 | 5/3 | -4/3 | -1 | 1 | 0 | 2 |
|  $s_1$  | 0 | 0 | 5/3 | -1/3 | 0 | 0 | 1 | 3 |
|  $c_j - s_j$  | | 0 | -1/3-5/3M | -4/3+7/3M | M | 0 | 0 | 4-2M |

```

```

Terminal — python3 big_m_method.py

-----

Entering variable: m2      (cj-mj = -1/3-5/3M)

Minimum-ratio test (leaving variable):
BV      RHS      pivot col  ratio
m1      1        1/3        3
m2      2        5/3        6/5
s1      3        5/3        9/5
Leaving variable: m2
Pivot element: 5/3 at (m2, m2)

SIMPLEX TABLEAU - Iteration 2 (current Z = 18/5)
-----
| BV | cb | m1 | m2 | a1 | p1 | a2 | s1 | RHS |
-----
| m1 | 4 | 1 | 0 | 2/5 | 1/5 | -1/5 | 0 | 2/5 |
| m2 | 1 | 0 | 1 | -4/5 | -2/5 | 3/5 | 0 | 6/5 |
| s1 | 0 | 0 | 0 | 1 | 1 | -1 | 1 | 1 |
| cj - mj | 0 | 0 | 0 | -8/5+M | -1/5 | 1/5+M | 0 | 18/5 |
-----

Entering variable: p1      (cj-mj = -1/5)

Minimum-ratio test (leaving variable):
BV      RHS      pivot col  ratio
m1      2/5      1/5        3
m2      6/5      -2/5      --
s1      1        1        1
Leaving variable: s1
Pivot element: 1 at (s1, p1)

SIMPLEX TABLEAU - Iteration 3 (current Z = 17/5)
-----
| BV | cb | m1 | m2 | a1 | p1 | a2 | s1 | RHS |
-----
| m1 | 4 | 1 | 0 | 2/5 | 0 | 0 | -1/5 | 2/5 |
| m2 | 1 | 0 | 1 | -1/5 | 0 | 0 | 3/5 | 9/5 |
| p1 | 0 | 0 | 0 | 1 | 1 | -1 | 1 | 1 |
| cj - mj | 0 | 0 | 0 | -7/5+M | 0 | M | 1/5 | 17/5 |
-----

All reduced costs satisfy the optimality condition. STOP.

=====
OPTIMAL SOLUTION
=====
Decision variables:
m1* = 2/5 (0.4000)
m2* = 9/5 (1.8000)
Slack / surplus / artificial (all artificials must be 0):
s1 = 0
p1 = 1
a2 = 0
s1 = 0

Optimal objective value: Z* = 17/5 (3.4000)
=====

```

Part 2 — Transportation Problem (TP)

A transportation problem ships a homogeneous product from m sources (capacities a_i) to n destinations (demands b_j) at unit cost c_{ij} . Minimize the total shipping cost subject to supply and demand equalities, $x_{ij} \geq 0$.

The problem is balanced when total supply equals total demand. A basic feasible solution occupies $m + n - 1$ independent cells.

The two methods are applied one at a time: VAM builds an initial basic feasible solution; MODI tests optimality and improves the plan.

Selected problem (3 factories, 4 warehouses)

	W1	W2	W3	W4	Supply
F1	19	30	50	10	7
F2	70	30	40	60	9
F3	40	8	70	20	18
Demand	5	8	7	14	$34 = 34$

Balanced. Need $3+4-1 = 6$ basic cells. Objective: minimize total transportation cost.

(i) Vogel's Approximation Method (VAM)

Penalty of a row or column = (second-smallest cost) - (smallest cost) among remaining cells. A large penalty means that skipping the cheapest cell in that line is expensive.

At each step: compute penalties; pick the largest; allocate as much as possible to that line's cheapest cell; cross out the exhausted row or column; repeat.

Part 2(i) — VAM initial solution

Step	Max penalty	Cell	Qty	Cross out
1	col W2 = 22	F3 -> W2	8	W2
2	col W1 = 21	F1 -> W1	5	W1
3	row F3 = 50	F3 -> W4	10	F3
4	col W4 = 50	F1 -> W4	2	F1
5	col W4 = 60	F2 -> W4	2	W4
final	—	F2 -> W3	7	—

IBFS (6 basic cells) cost = 779

	W1	W2	W3	W4
F1	5	—	—	2
F2	—	—	7	2
F3	—	8	—	10

$$Z_{\text{VAM}} = 5 \cdot 19 + 2 \cdot 10 + 7 \cdot 40 + 2 \cdot 60 + 8 \cdot 8 + 10 \cdot 20 = 779$$

This IBFS is not guaranteed optimal. MODI (next) tests and improves it.

Command: `python3 transportation.py --method vam`

Source file transportation.py. VAM: function vam(). MODI: function modi(). Run one method at a time.

```

1  """
2  Transportation Problem - VAM then MODI
3  =====
4  (i) Vogel's Approximation Method (VAM) -> initial basic feasible solution
5  (ii) MODI (Modified Distribution / UV) -> optimality test and improvement
6
7  Classic balanced 3-source, 4-destination problem (Sharma / Gupta / Taha):
8
9          W1    W2    W3    W4    Supply
10         F1    19    30    50    10    7
11         F2    70    30    40    60    9
12         F3    40    8     70    20    18
13         Demand 5     8     7    14    24
14
15
16 from copy import deepcopy
17 from typing import List, Optional, Sequence, Tuple
18
19 INF = 10**9
20 EPS = 1e-9
21
22
23 def print_matrix(
24     cost: Sequence[Sequence[float]],
25     supply: Sequence[float],
26     demand: Sequence[float],
27     alloc: Optional[Sequence[Sequence[Optional[float]]]] = None,
28     title: str = "",
29     sources: Optional[Sequence[str]] = None,
30     dests: Optional[Sequence[str]] = None,
31 ) -> None:
32     m, n = len(supply), len(demand)
33     sources = list(sources) if sources else [f"S{i+1}" for i in range(m)]
34     dests = list(dests) if dests else [f"D{j+1}" for j in range(n)]
35
36     headers = [""].copy() + list(dests) + ["Supply"]
37     rows = []
38     for i in range(m):
39         cells = [sources[i]]
40         for j in range(n):
41             c = cost[i][j]
42             if alloc is None:
43                 cells.append(str(c))
44             else:
45                 a = alloc[i][j]
46                 mark = "-" if a is None else (str(int(a) if a == int(a) else a))
47                 cells.append(f"{c} [{mark}]")
48             a = supply[i]
49             cells.append(str(int(a) if a == int(a) else a))
50             rows.append(cells)
51     dem = ["Demand"] + [str(int(d) if d == int(d) else d) for d in demand] + [""]
52     rows.append(dem)
53
54     widths = [len(h) for h in headers]
55     for row in rows:
56         for k, cell in enumerate(row):
57             widths[k] = max(widths[k], len(str(cell)))
58
59     def hline() -> str:
60         return "+" + "+" * (n + 1) + "+"
61
62     def prow(cells: Sequence[str]) -> str:

```

```

63     return "|" + "|".join(f" {c:{w}} " for c, w in zip(cells, widths)) + "|"
64
65     if title:
66         print(f"\n{title}")
67     print(hline())
68     print(prov(headers))
69     print(hline())
70     for row in rows[:-1]:
71         print(prov(row))
72     print(hline())
73     print(prov(rows[-1]))
74     print(hline())
75
76
77 def total_cost(cost, alloc) -> float:
78     tot = 0.0
79     for i in range(len(alloc)):
80         for j in range(len(alloc[0])):
81             a = alloc[i][j]
82             if a is not None and a > 0:
83                 tot += a * cost[i][j]
84     return tot
85
86
87 def count_basic(alloc) -> int:
88     return sum(1 for row in alloc for a in row if a is not None)
89
90
91 # =====
92 # (i) Vogel's Approximation Method
93 # =====
94 def row_penalty(cost_row: Sequence[float], active_cols: Sequence[int]) -> float:
95     vals = sorted(cost_row[j] for j in active_cols)
96     if not vals:
97         return -1
98     if len(vals) == 1:
99         return vals[0]
100     return vals[1] - vals[0]
101
102
103 def col_penalty(cost, active_rows, j) -> float:
104     vals = sorted(cost[i][j] for i in active_rows)
105     if not vals:
106         return -1
107     if len(vals) == 1:
108         return vals[0]
109     return vals[1] - vals[0]
110
111
112 def vam(
113     cost: List[List[float]],
114     supply: List[float],
115     demand: List[float],
116     sources: Sequence[str],
117     dests: Sequence[str],
118 ) -> List[List[Optional[float]]]:
119     m, n = len(supply), len(demand)
120     rem_s = list(supply)
121     rem_d = list(demand)
122     alloc: List[List[Optional[float]]] = [[None] * n for _ in range(m)]
123     active_r = set(range(m))
124     active_c = set(range(n))

```

```

125     step = 0
126
127     print("\n" + "=" * 72)
128     print("(1) VOGEL'S APPROXIMATION METHOD - Initial Basic Feasible Solution")
129     print("=" * 72)
130     print(" At each step:")
131     print(" 1. Penalty of a row/column = (2nd lowest cost) - (lowest cost)")
132     print(" 2. Choose the row or column with the largest penalty")
133     print(" 3. Allocate as much as possible to its cheapest available cell")
134     print(" 4. Cross out the exhausted row or column and repeat")
135
136     while active_r and active_c:
137         step += 1
138         r_pen = {i: row_penalty(cost[i], list(active_c)) for i in active_r}
139         c_pen = {j: col_penalty(cost, list(active_r), j) for j in active_c}
140
141         print(f"==== VAM Step {step} ===")
142         print(" Remaining supply:", {sources[i]: rem_s[i] for i in sorted(active_r)})
143         print(" Remaining demand:", {dests[j]: rem_d[j] for j in sorted(active_c)})
144         print(" Row penalties: ", {sources[i]: r_pen[i] for i in sorted(active_r)})
145         print(" Column penalties:", {dests[j]: c_pen[j] for j in sorted(active_c)})
146
147         # Highest penalty; ties broken by cheapest cell in that line.
148         best_kind = None # 'row' or 'col'
149         best_idx = None
150         best_pen = -1.0
151         best_cheap = INF
152
153         for i, p in r_pen.items():
154             cheap = min(cost[i][j] for j in active_c)
155             if p > best_pen or (p == best_pen and cheap < best_cheap):
156                 best_pen, best_cheap = p, cheap
157                 best_kind, best_idx = "row", i
158         for j, p in c_pen.items():
159             cheap = min(cost[i][j] for i in active_r)
160             if p > best_pen or (p == best_pen and cheap < best_cheap):
161                 best_pen, best_cheap = p, cheap
162                 best_kind, best_idx = "col", j
163
164         if best_kind == "row":
165             i = best_idx
166             j = min(active_c, key=lambda jj: cost[i][jj])
167             print(f" Largest penalty is ROW {sources[i]} = {best_pen}")
168             print(f" Cheapest cell in this row: ({sources[i]}, {dests[j]}) cost {cost[i][j]}")
169         else:
170             j = best_idx
171             i = min(active_r, key=lambda ii: cost[ii][j])
172             print(f" Largest penalty is COLUMN {dests[j]} = {best_pen}")
173             print(f" Cheapest cell in this column: ({sources[i]}, {dests[j]}) cost {cost[i][j]}")
174
175         qty = min(rem_s[i], rem_d[j])
176         alloc[i][j] = (alloc[i][j] or 0) + qty
177         rem_s[i] -= qty
178         rem_d[j] -= qty
179         print(f" Allocate {qty} units to ({sources[i]}, {dests[j]})")
180
181         # Cross out exhausted lines. If both become zero, drop one and keep
182         # the other as a degenerate basic cell of 0 (needed for m+n-1).
183         row_out = rem_s[i] <= EPS
184         col_out = rem_d[j] <= EPS
185         if row_out and col_out and len(active_r) + len(active_c) > 2:
186             # Degeneracy: keep the row, drop the column (0 already allocated).

```

```

187         active_c.discard(j)
188         print(f" Both {sources[i]} and {dests[j]} exhausted (degenerate). Cross out {dests[j]}.")
189     elif row_out:
190         active_r.discard(i)
191         print(f" Cross out {sources[i]} (supply exhausted).")
192     elif col_out:
193         active_c.discard(j)
194         print(f" Cross out {dests[j]} (demand exhausted).")
195
196     if len(active_r) == 1 and len(active_c) == 1:
197         i = next(iter(active_r))
198         j = next(iter(active_c))
199         qty = min(rem_s[i], rem_d[j])
200         if qty > EPS or alloc[i][j] is None:
201             alloc[i][j] = (alloc[i][j] or 0) + qty
202             rem_s[i] -= qty
203             rem_d[j] -= qty
204             print(f" Final allocation: {qty} units to ({sources[i]}, {dests[j]}).")
205         active_r.clear()
206         active_c.clear()
207
208     print_matrix(cost, supply, demand, alloc, "VAM allocation [cost, quantity]", sources, dests)
209     c = total_cost(cost, alloc)
210     print(f"\n Number of basic cells: {count_basic(alloc)} (need m+n-1 = {m}+{n}-1 = {m+n-1})")
211     print(f" Initial transportation cost (VAM): {c:.0f}")
212     return alloc
213
214
215 # =====
216 # (ii) MODI (Modified Distribution / UV method)
217 # =====
218 def _occupied(alloc) -> List[Tuple[int, int]]:
219     return [(i, j) for i, row in enumerate(alloc) for j, a in enumerate(row) if a is not None]
220
221
222 def _ensure_non_degenerate(alloc, m, n) -> None:
223     """Add 0-basic cells until there are m+n-1 independents."""
224     needed = m + n - 1
225     while count_basic(alloc) < needed:
226         placed = False
227         for i in range(m):
228             for j in range(n):
229                 if alloc[i][j] is not None:
230                     continue
231                 # Independent if it does not close a cycle with current basics.
232                 trial = deepcopy(alloc)
233                 trial[i][j] = 0.0
234                 if _find_cycle(trial, i, j) is None:
235                     alloc[i][j] = 0.0
236                     print(f" Degeneracy: add a 0-basic cell at ({i}, {j})")
237                     placed = True
238                     break
239             if placed:
240                 break
241         if not placed:
242             break
243
244
245 def _find_cycle(alloc, si, sj) -> Optional[List[Tuple[int, int]]]:
246     """
247     Closed loop through occupied cells starting at the entering cell (si, sj).
248     Moves alternate horizontal / vertical. Returns vertices in order, or None.

```

```

249     """
250     occ = set(_occupied(alloc))
251     occ.add((si, sj))
252
253     def neighbors(i, j, horiz: bool):
254         if horiz:
255             return [(i, jj) for ii, jj in occ if ii == i and jj != j]
256             return [(ii, j) for ii, jj in occ if jj == j and ii != i]
257
258     def search(path, used, horiz: bool):
259         i, j = path[-1]
260         for nmt in neighbors(i, j, horiz):
261             if nmt == (si, sj):
262                 # Even-length cycle: path currently has an even number of vertices.
263                 if len(path) >= 4 and len(path) % 2 == 0:
264                     return path
265                 continue
266             if nmt in used:
267                 continue
268             used.add(nmt)
269             got = search(path + [nmt], used, not horiz)
270             if got is not None:
271                 return got
272             used.remove(nmt)
273         return None
274
275     # Try a horizontal first step, then a vertical first step.
276     for first_horiz in (True, False):
277         found = search([(si, sj)], {(si, sj)}, first_horiz)
278         if found is not None:
279             return found
280     return None
281
282
283 def _solve_uv(cost, alloc) -> Tuple[List[Optional[float]], List[Optional[float]]]:
284     m, n = len(alloc), len(alloc[0])
285     u = [None] * m
286     v = [None] * n
287     basics = _occupied(alloc)
288     u[0] = 0.0 # arbitrary origin
289     changed = True
290     guard = 0
291     while changed and guard < m * n + 5:
292         changed = False
293         guard += 1
294         for i, j in basics:
295             if u[i] is not None and v[j] is None:
296                 v[j] = cost[i][j] - u[i]
297                 changed = True
298             elif v[j] is not None and u[i] is None:
299                 u[i] = cost[i][j] - v[j]
300                 changed = True
301     return u, v
302
303
304 def modi(
305     cost: List[List[float]],
306     supply: List[float],
307     demand: List[float],
308     alloc: List[List[Optional[float]]],
309     sources: Sequence[str],
310     dests: Sequence[str],

```

```

transportation.py — Python source

211 ) -> List[List[Optional[float]]]:
212     m, n = len(supply), len(demand)
213     alloc = deepcopy(alloc)
214
215     print("\n" + "=" * 72)
216     print("(ii) MODI METHOD — Optimality test and iterative improvement")
217     print("=" * 72)
218     print(" For every basic cell (i, j):    u_i + v_j = c_ij")
219     print(" For every non-basic cell:      d_ij = c_ij - u_i - v_j")
220     print(" MIN problem is optimal when every d_ij >= 0.")
221     print(" If some d_ij < 0, enter the most negative cell, form a closed")
222     print(" loop with basic cells, and shift theta = min of (-) positions.")
223
224     iteration = 0
225     while True:
226         iteration += 1
227         _ensure_non_degenerate(alloc, m, n)
228         u, v = _solve_uv(costs, alloc)
229
230         print(f"\n{'=' * 60}")
231         print(f"MODI Iteration {iteration}")
232         print(f"{'=' * 60}")
233         print_matrix(costs, supply, demand, alloc, "Current allocation [cost (quantity)]", sources, dests)
234
235         def _num(x):
236             if x is None:
237                 return None
238             return int(x) if abs(x - round(x)) < 1e-9 else x
239
240         print(" Dual variables (set u1 = 0):")
241         print("   u = ", {sources[i]: _num(u[i]) for i in range(m)})
242         print("   v = ", {dests[j]: _num(v[j]) for j in range(n)})
243
244         if any(x is None for x in u + v):
245             print(" Could not determine all u_i, v_j (basis not a tree).")
246             break
247
248         d = {[None] * n for _ in range(m)}
249         most_neg = 0.0
250         enter = None
251         print("\n Opportunity costs d_ij = c_ij - u_i - v_j (blank = basic):")
252         headers = [""] + list(dests)
253         rows = []
254         for i in range(m):
255             row = [sources[i]]
256             for j in range(n):
257                 if alloc[i][j] is not None:
258                     row.append("basic")
259                     continue
260                 dij = costs[i][j] - u[i] - v[j]
261                 d[i][j] = dij
262                 row.append(f"{dij:g}")
263                 if dij < most_neg - 1e-12:
264                     most_neg = dij
265                     enter = (i, j)
266             rows.append(row)
267
268         widths = [max(len(headers[k]), max(len(r[k]) for r in rows)) for k in range(n + 1)]
269         line = "" + "".join("-" * (w + 2) for w in widths) + "="
270         print(line)
271         print("|" + "|" + "".join(f" {headers[k]:^{widths[k]}} " for k in range(n + 1)) + "|")
272         print(line)

```

```

transportation.py — Python source

273     for x in rows:
274         print(" " + " ".join("%" % (x[k] / widths[k]) for k in range(n + 1)) + " ")
275     print(line)
276
277     if enter is None:
278         print("\n All dij >= 0. The allocation is OPTIMAL.")
279         break
280
281     ei, ej = enter
282     print(f"\n Most negative opportunity cost: d({sources[ei]}, {dests[ej]}) = {most_neg:g}")
283     print(f" Entering cell: ({sources[ei]}, {dests[ej]})")
284
285     cycle = _find_cycle(alloc, ei, ej)
286     if cycle is None:
287         print(" No closed loop found; cannot improve. Stopping.")
288         break
289
290     labels = [{"sources[i]}, {dests[j]}"] for i, j in cycle
291     signs = ["+" if k % 2 == 0 else "-" for k in range(len(cycle))]
292     print(" Closed loop (alternating + / -):")
293     print(" ", " -> ".join("%(lab)[%(sg)]" for lab, sg in zip(labels, signs)))
294
295     minus_cells = [cycle[k] for k in range(len(cycle)) if k % 2 == 1]
296     theta = min(alloc[i][j] for i, j in minus_cells)
297     print(f" theta = min of (-) allocations = {theta:g}")
298
299     for k, (i, j) in enumerate(cycle):
300         if k % 2 == 0:
301             alloc[i][j] = (alloc[i][j] or 0) + theta
302         else:
303             alloc[i][j] = alloc[i][j] - theta
304             if abs(alloc[i][j]) <= EPS:
305                 alloc[i][j] = None # leaves the basis
306
307     print(f" Shifts {theta:g} units around the loop.")
308     print(f" Cost after this iteration: {total_cost(cost, alloc):.0f}")
309
310     z = total_cost(cost, alloc)
311     print("\n" + "=" * 72)
312     print("OPTIMAL SHIPMENT PLAN")
313     print("=" * 72)
314     print_matrix(cost, supply, demand, alloc, "Optimal allocation [cost (quantity)]", sources, dests)
315     print("\n Positive shipments:")
316     for i in range(m):
317         for j in range(n):
318             a = alloc[i][j]
319             if a is not None and a > EPS:
320                 print(f" {sources[i]} -> {dests[j]} : {a:g} units (cost {cost[i][j]} each)")
321     print(f"\n Minimum total transportation cost Z* = {z:.0f}")
322     print("=" * 72)
323     return alloc
324
325
326 def problem_data():
327     cost = [
328         [19, 30, 50, 10],
329         [70, 30, 40, 60],
330         [40, 8, 70, 20],
331     ]
332     supply = [7, 9, 18]
333     demand = [8, 8, 7, 14]
334     sources = ["F1", "F2", "F3"]

```

```

435     dests = ["W1", "W2", "W3", "W4"]
436     return cost, supply, demand, sources, dests
437
438
439 def print_problem(cost, supply, demand, sources, dests) -> None:
440     print("\n\nASSIGNMENT — TRANSPORTATION PROBLEM (VAM + MODI)\n")
441     print("=" * 72)
442     print("PROBLEM STATEMENT")
443     print("=" * 72)
444     print(" Three factories ship to four warehouses.")
445     print(" Unit transportation costs, supplies and demands:")
446     print_matrix(cost, supply, demand, None, "", sources, dests)
447     print(f" Total supply = {sum(supply)} = total demand = {sum(demand)} (balanced)")
448     print(" Objective: minimize total transportation cost.")
449
450
451 def main(method: str = "both") -> None:
452     """
453     method: 'vam' | 'modi' | 'both'
454     MODI always starts from the VAM initial basic feasible solution.
455     """
456     cost, supply, demand, sources, dests = problem_data()
457     print_problem(cost, supply, demand, sources, dests)
458
459     ibfs = vam(cost, supply, demand, sources, dests)
460     if method == "vam":
461         print("\n (MODI skipped — run with --method modi or --method both to test optimality.)")
462         return
463     modi(cost, supply, demand, ibfs, sources, dests)
464
465
466 if __name__ == "__main__":
467     import argparse
468
469     parser = argparse.ArgumentParser(description="Solve the transportation problem with VAM and/or MODI.")
470     parser.add_argument(
471         "--method",
472         choices=["vam", "modi", "both"],
473         default="both",
474         help="vam: initial solution only; modi/both: VAM then MODI optimality test",
475     )
476     args = parser.parse_args()
477     main(args.method)

```

Captured terminal output of the program (screenshot-style).

```

Terminal — python3 transportation.py --method vam

ASSIGNMENT - TRANSPORTATION PROBLEM (VAM + MODI)

=====
PROBLEM STATEMENT
=====
Three factories ship to four warehouses.
Unit transportation costs, supplies and demands:
-----
|      | W1 | W2 | W3 | W4 | Supply |
|-----|
| F1   | 19 | 30 | 50 | 10 | 7      |
| F2   | 70 | 30 | 40 | 60 | 9      |
| F3   | 40 | 8  | 70 | 20 | 18     |
|-----|
| Demand | 5 | 8 | 7 | 14 |        |
|-----|

Total supply = 34 = total demand = 34 (balanced)
Objective: minimize total transportation cost.

=====
(a) VOGEL'S APPROXIMATION METHOD - Initial Basic Feasible Solution
=====

At each step:
1. Penalty of a row/column = (2nd lowest cost) - (lowest cost)
2. Choose the row or column with the largest penalty
3. Allocate as much as possible to its cheapest available cell
4. Cross out the exhausted row or column and repeat

--- VAM Step 1 ---
Remaining supply: {'F1': 7, 'F2': 9, 'F3': 18}
Remaining demand: {'W1': 5, 'W2': 8, 'W3': 7, 'W4': 14}
Row penalties:    {'F1': 9, 'F2': 10, 'F3': 12}
Column penalties: {'W1': 21, 'W2': 22, 'W3': 10, 'W4': 10}
Largest penalty is COLUMN W2 = 22
Cheapest cell in this column: (F3, W2) cost 8
Allocate 8 units to (F3, W2)
Cross out W2 (demand exhausted).

--- VAM Step 2 ---
Remaining supply: {'F1': 7, 'F2': 9, 'F3': 10}
Remaining demand: {'W1': 5, 'W3': 7, 'W4': 14}
Row penalties:    {'F1': 9, 'F2': 20, 'F3': 20}
Column penalties: {'W1': 21, 'W3': 10, 'W4': 10}
Largest penalty is COLUMN W1 = 21
Cheapest cell in this column: (F1, W1) cost 19
Allocate 5 units to (F1, W1)
Cross out W1 (demand exhausted).

--- VAM Step 3 ---
Remaining supply: {'F1': 2, 'F2': 9, 'F3': 10}
Remaining demand: {'W3': 7, 'W4': 14}
Row penalties:    {'F1': 40, 'F2': 20, 'F3': 50}
Column penalties: {'W3': 10, 'W4': 10}
Largest penalty is ROW F3 = 50
Cheapest cell in this row: (F3, W4) cost 20
Allocate 10 units to (F3, W4)
Cross out F3 (supply exhausted).

--- VAM Step 4 ---
Remaining supply: {'F1': 2, 'F2': 9}
Remaining demand: {'W3': 7, 'W4': 4}
Row penalties:    {'F1': 40, 'F2': 20}
Column penalties: {'W3': 10, 'W4': 50}
Largest penalty is COLUMN W4 = 50

```

```

Terminal — python3 transportation.py --method vam

Cheapest cell in this column: (F1, W4) cost 10
Allocate 2 units to (F1, W4)
Cross out F1 (supply exhausted).

--- VAM Step 5 ---
Remaining supply: {'F2': 9}
Remaining demand: {'W3': 7, 'W4': 2}
Row penalties: {'F2': 20}
Column penalties: {'W3': 40, 'W4': 60}
Largest penalty is COLUMN W4 = 60
Cheapest cell in this column: (F2, W4) cost 60
Allocate 2 units to (F2, W4)
Cross out W4 (demand exhausted).
Final allocation: 7 units to (F2, W4)

VAM allocation (cost (quantity))
-----
|      | W1 | W2 | W3 | W4 | Supply |
|-----|
| F1 | 19 [5] | 30 [-] | 40 [-] | 10 [2] | 7 |
| F2 | 70 [-] | 30 [-] | 40 [7] | 60 [2] | 9 |
| F3 | 40 [-] | 8 [8] | 70 [-] | 20 [10] | 18 |
|-----|
| Demand | 5 | 8 | 7 | 14 | |
|-----|

Number of basic cells: 6 (need m*n-1 = 3*4-1 = 6)
Initial transportation cost (VAM): 779

(MODI skipped - run with --method modi or --method both to test optimality.)

```

Captured terminal output of the program (screenshot-style).

```

Terminal — python3 transportation.py --method modi

ASSIGNMENT - MODI METHOD (starts from the VAM IBFS of cost 779)

(ii) MODI METHOD - Optimality test and iterative improvement
=====
For every basic cell (i, j):  $u_i + v_j = c_{ij}$ 
For every non-basic cell:  $d_{ij} = c_{ij} - u_i - v_j$ 
MIN problem is optimal when every  $d_{ij} \geq 0$ .
If some  $d_{ij} < 0$ , enter the most negative cell, form a closed
loop with basic cells, and shift theta = min of (-) positions.

=====
MODI Iteration 1
=====

Current allocation [cost (quantity)]
-----
|   | W1 | W2 | W3 | W4 | Supply |
-----
| F1 | 19 [5] | 30 [-] | 50 [-] | 10 [2] | 7 |
| F2 | 70 [-] | 30 [-] | 40 [7] | 60 [2] | 9 |
| F3 | 40 [-] | 8 [8] | 70 [-] | 20 [10] | 18 |
-----
| Demand | 5 | 8 | 7 | 14 | |
-----

Dual variables (set  $u_1 = 0$ ):
u = {'F1': 0, 'F2': 50, 'F3': 10}
v = {'W1': 19, 'W2': -2, 'W3': -10, 'W4': 10}

Opportunity costs  $d_{ij} = c_{ij} - u_i - v_j$  (blank = basic):
-----
|   | W1 | W2 | W3 | W4 |
-----
| F1 | basic | 32 | 60 | basic |
| F2 | 1 | -18 | basic | basic |
| F3 | 11 | basic | 70 | basic |
-----

Most negative opportunity cost: d(F2, W2) = -18
Entering cell: (F2, W2)
Closed loop (alternating + / -):
(F2, W2)[+] -> (F2, W4)[-] -> (F3, W4)[+] -> (F3, W2)[-]
theta = min of (-) allocations = 2
Shift 2 units around the loop.
Cost after this iteration: 748

=====
MODI Iteration 2
=====

Current allocation [cost (quantity)]
-----
|   | W1 | W2 | W3 | W4 | Supply |
-----
| F1 | 19 [5] | 30 [-] | 50 [-] | 10 [2] | 7 |
| F2 | 70 [-] | 30 [2] | 40 [7] | 60 [-] | 9 |
| F3 | 40 [-] | 8 [6] | 70 [-] | 20 [12] | 18 |
-----
| Demand | 5 | 8 | 7 | 14 | |
-----

Dual variables (set  $u_1 = 0$ ):
u = {'F1': 0, 'F2': 32, 'F3': 10}
v = {'W1': 19, 'W2': -2, 'W3': 8, 'W4': 10}

Opportunity costs  $d_{ij} = c_{ij} - u_i - v_j$  (blank = basic):

```

Terminal — python3 transportation.py --method modi

```

-----
|   | W1 | W2 | W3 | W4 |
-----
| F1 | basic | 32 | 42 | basic |
| F2 | 19 | basic | basic | 18 |
| F3 | 11 | basic | 52 | basic |
-----

All d_ij >= 0. The allocation is OPTIMAL.

=====
OPTIMAL SHIPMENT PLAN
=====

Optimal allocation [cost (quantity)]

-----
|   | W1 | W2 | W3 | W4 | Supply |
-----
| F1 | 19 [5] | 30 [-] | 50 [-] | 10 [2] | 7 |
| F2 | 70 [-] | 30 [2] | 40 [7] | 60 [-] | 9 |
| F3 | 40 [-] | 8 [6] | 70 [-] | 20 [12] | 18 |
-----
| Demand | 5 | 8 | 7 | 14 | |
-----

Positive shipments:
F1 -> W1 : 5 units (cost 19 each)
F1 -> W4 : 2 units (cost 10 each)
F2 -> W2 : 2 units (cost 30 each)
F2 -> W3 : 7 units (cost 40 each)
F3 -> W2 : 6 units (cost 8 each)
F3 -> W4 : 12 units (cost 20 each)

Minimum total transportation cost Z* = 743
=====

```